Unsupervised learning

Nathanaël Carraz Rakotonirina

Mathématiques Informatique et Statistique Appliquées (MISA)
Université d'Antananarivo

Principal Component Analysis

Clustering

Dimensionality reduction

Goal of dimension reduction

Find a projection of the high dimensional data $\boldsymbol{x} \in \mathbb{R}^D$ to a low dimensional subspace $\boldsymbol{z} \in \mathbb{R}^L$ ($L < D$), such that the low dimensional representation is a "good approximation" to the original data.

$\boldsymbol{z}$ is called the **latent vector** because its values are latent or "hidden" or not observed in the data. The collection of these latent variables are called the **latent factors**. Dimensionality reduction methods are used for :

- visualization
- data representation (noise reduction, pre-processing, hidden structure in data)

Model

For PCA, the projection is linear and orthogonal. We project or encode $\boldsymbol{x}$ to obtain $z = \boldsymbol{W}^\top x$, then unproject or decode $\boldsymbol{z}$ to get $\hat{\boldsymbol{x}} = \boldsymbol{W}\boldsymbol{z}$ such that $\hat{\boldsymbol{x}}$ is close to $\boldsymbol{x}$ in l_2 distance.

$$\text{encode} : \mathbb{R}^D \to \mathbb{R}^L \qquad \boldsymbol{x} \mapsto \boldsymbol{z} = \boldsymbol{W}^\top \boldsymbol{x}$$

$$\text{decode} : \mathbb{R}^L \to \mathbb{R}^D \qquad \boldsymbol{z} \mapsto \hat{\boldsymbol{x}} = \boldsymbol{W}\boldsymbol{z}$$

The **reconstruction error** is:

$$\begin{aligned} L(\boldsymbol{W}) &= \frac{1}{N}\sum_{i=1}^{N} ||\boldsymbol{x}_i - \text{decode}(\text{encode}(\boldsymbol{x}_i))||^2 \\ &= \frac{1}{N}\sum_{i=1}^{N} ||\boldsymbol{x}_i - \boldsymbol{W}\boldsymbol{z}_i||^2 = \frac{1}{N}\sum_{i=1}^{N} ||\boldsymbol{x}_i - \boldsymbol{W}\boldsymbol{W}^\top \boldsymbol{x}_i||^2 \end{aligned}$$

Solution

We assume the data is centered (mean is zero).
We want to minimize the objective to the constraint that $\boldsymbol{W}$ is an orthogonal matrix.
The optimal solution is

$$\hat{\boldsymbol{W}} = U_L$$

where U_L contains the L eigenvectors with largest eigenvalues of the empirical covariance matrix : $\hat{\boldsymbol{\Sigma}} = \frac{1}{N}\boldsymbol{X}_c^\top \boldsymbol{X}_c$ where $\boldsymbol{X}_c$ is a centered version of the design matrix

It can be shown that minimizing the reconstruction error is equivalent to maximizing the variance of the projected data

PCA in a nutshell

1. Normalize the data
2. Compute the covariance matrix
3. Find the eigenvalue decompositon of the covariance matrix
4. Order the eigenvalues by decreasing order
5. Get the projection matrix whose columns are the L eigenvectors associated with the L largest eigenvalues
6. Project your data using the projection matrix

How to choose the number of latent dimensions ?

We cannot use the reconstruction error on a validation set as a way to select the best unsupervised model.
We could do the following.

- Scree plots
- Elbow trick (profile likelihood)

Computational issues

- When $D > N$, it is faster to use the Gram matrix $\boldsymbol{X}\boldsymbol{X}^\top$ instead.
- Computing the SVD decomposition of $\boldsymbol{X}$ is equivalent to computing the eigendecomposition of the covariance matrix but SVD is more numerically stable.
- Consider using randomized SVD for very high dimensional problems.

Beyond PCA

- Variants : Probabilistic PCA, kernel PCA
- Other dimensionality reduction methods : ISOMAP, LLE, MVU, SNE, t-SNE
- Neural networks approaches : auto-encoders, embeddings (word embeddings)

Principal Component Analysis

Clustering

Clustering

Clustering is an unsupervised learning approach. The inputs are $\mathcal{D} = \{\boldsymbol{x}_1, ..., \boldsymbol{x}_N\}$ where $\boldsymbol{x}_i \in \mathbb{R}^D$. Data samples are unlabeled.

Goal

The goal is to assign similar data points to the same cluster.

Applications of clustering:

- resembling genes
- image segmentation
- customer segmentation
- social sub-networks
- user profile

Hierarchical agglomerative clustering

- The input to the algorithm is an $N \times N$ dissimilarity matrix D ($D_{ij} > 0$ is the distance between groups i and j)
- Groups with small dissimilarity are grouped together in a hierarchical fashion.
- The output is a binary tree called **dendogram**.
- By cutting this tree at different heights, we can induce a different number of (nested) clusters.

The algorithm:

1. Initialize each sample as a cluster : $C_i = \{i\}$ for $i = 1$ to N
2. Initialize set of clusters available for merging : $S = \{C_1, ..., C_N\}$
3. Compute the dissimilarity matrix D according to some distance
4. Repeat until all samples are merged in one cluster
   - Pick the two closest clusters: (C_j, C_k)
   - Merge them into a new cluster : $C_l = C_j \cup C_k$
   - Remove C_j and C_k from the available clusters : $S = S \setminus \{C_j, C_k\}$
   - Add C_l to the available clusters : $S = S \cup C_l$
   - Update the dissimilarity matrix by adding the distance between C_l and the other available clusters.

How to measure dissimilarity between groups

- **Single link**: also called nearest neighbor clustering, the distance between two clusters is defined as the distance between the two closest members of each group:

$$d_{\min}(G, H) = \min\{d(i,j), i \in G, j \in H\}$$

- **Complete link**: also called furthest neighbor clustering, the distance between two clusters is defined as the distance between the two most distant pairs:

$$d_{\max}(G, H) = \max\{d(i,j), i \in G, j \in H\}$$

- **Average link**: the distance between two clusters is the average distance between all pairs

$$d_{\text{avg}}(G, H) = \frac{1}{n_G n_H} \sum_{i \in G} \sum_{j \in H} d(i,j)$$

- **Ward's distance**: the distance between two clusters is the distance between centers:

$$d_{\text{avg}}(G, H) = \sqrt{\frac{n_G n_H}{n_G + n_H}} d(\mu_G, \mu_H)$$

K means clustering

It computes similarity in terms of Euclidean distance to learned cluster centers $\mu_k \in \mathbb{R}^D$. Unlike hierachical agglomerative clustering:

- It is faster ($O(N)$ while average link HAC is $O(N^3)$).
- It optimizes a well-defined cost-function (it is a model and not just an algorithm).

The algorithm:

1. Initialize cluster centroids $\boldsymbol{\mu}_1, .., \boldsymbol{\mu}_K \in \mathbb{R}^D$ randomly
2. Repeat until convergence
   - Assign each data point $\boldsymbol{x}_i$ to the closest centeroid: $z_i = \arg\min_k ||\boldsymbol{x}_i - \boldsymbol{\mu}_k||_2^2$ (z_i is the index of the cluster assigned to $\boldsymbol{x}_i$)
   - Update the cluster centroids to the average of the points assigned to them : $\boldsymbol{\mu}_j = \frac{1}{n_j} \sum_{i:z_i=j} \boldsymbol{x}_i$

The loss can be formulated as

$$J(\boldsymbol{M}, \boldsymbol{Z}) = \sum_{i=1}^{N} ||\boldsymbol{x}_i - \boldsymbol{\mu}_{z_i}||^2 = ||\boldsymbol{X} - \boldsymbol{Z}\boldsymbol{M}^\top||_F^2$$

where $\boldsymbol{X} \in \mathbb{R}^{N \times D}$, $\boldsymbol{Z} \in [0,1]^{N \times K}$, $\boldsymbol{M} \in \mathbb{R}^{D \times K}$ (its columns are the clusters $\boldsymbol{\mu}_j$)

Limitations

- The loss is not convex thus it may converge to a local minimum.
- Different initializations can lead to different clusters (different local minima).

Solution

It is run many times using different random initial values for the cluster centroids, and then, pick the best solution. There are other variants.

How to choose K ?

We cannot use the reconstruction error on a validation set as a way to select the best unsupervised model (like PCA).
It is difficult to asses the number of clusters because of a lack of a ground truth to compare with.

- Manually fixed a priori
- Elbow trick
- Ad-hoc metrics (silhouette score)

Explore further

Other variants of K-means

- K-means++
- K-medoids
- faster K-means

Other clustering methods

- Clustering with mixture models
- Spectral clustering
- DBScan